

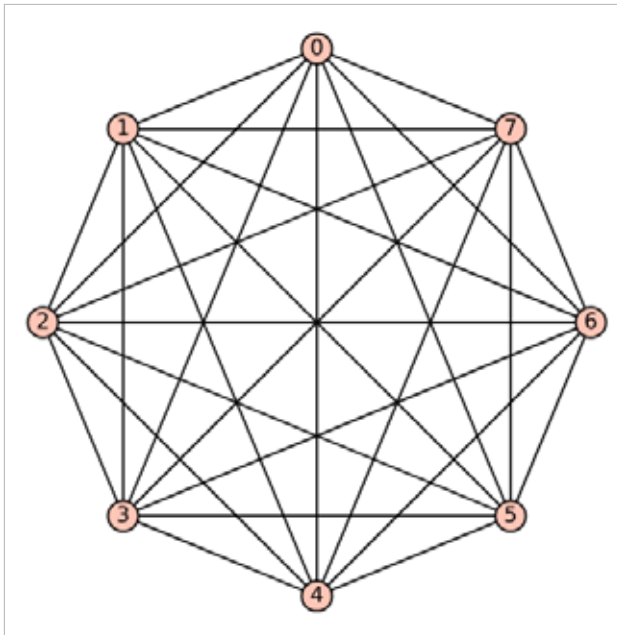


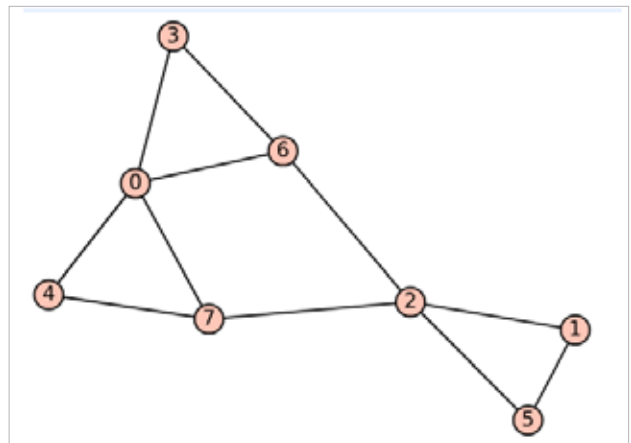
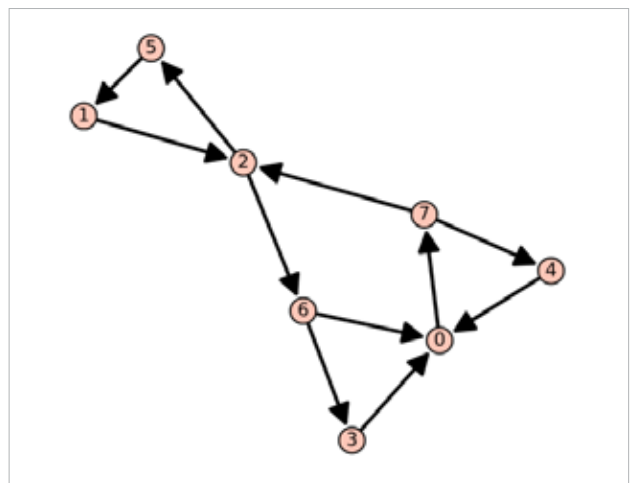
Figure 2: The undirected complete graph on 8 vertices

graph on 8 vertices), as shown in Figure 2. Why does this happen instead of producing distinct graphs each time? To answer this, we need to understand the purpose of the two input parameters given to the method `RandomGNP()`. The first parameter defines the number of vertices in the randomly generated graph (8 in this case). The second parameter defines the probability with which an edge is created between each pair of vertices, with a range between 0 and 1, inclusive. In the example above, the probability is set to 1, resulting in a 100% chance of generating an edge between each pair of vertices. Thus, an edge is created between every pair of vertices, leading to the generation of a complete graph. What graph will be displayed if the method `RandomGNP()` is called as: `'graphs.RandomGNP(8, 0)'`? Please give it a try before you proceed any further. I will provide the answer at the end of this article.

Now, let us change the second parameter so that the probability of creating an edge between each pair of vertices is 0.3. The program `'rgraph13.sagews'` shown below generates a random graph (line numbers have been included for clarity in the explanation).

```
1. rgraph = graphs.RandomGNP(8, 0.3)
2. rgraph.show( )
3. sgraph = rgraph.strong_orientation( )
4. sgraph.show( )
```

Now, let us go through the code for better understanding. Line 1 creates a random undirected graph `rgraph` using the `RandomGNP()` method from SageMath's `graphs` library. The first parameter, 8, specifies the number of vertices in the

Figure 3: The random graph *Rgraph_13*Figure 4: A strongly connected orientation of the graph *Rgraph_13*

graph. The second parameter, 0.3, specifies the probability of edge creation between any pair of vertices. Line 2 displays the random graph generated. Figure 3 shows a random graph called *Rgraph_13* generated in one of the executions of the above code. Keep in mind that the graph generated with each execution of the code will almost certainly be different from the one generated previously. Also, the chances of you getting a graph isomorphic to *Rgraph_13* are near zero.

Line 3 of the code converts the undirected graph `rgraph` into a directed graph `sgraph` using the `strong_orientation()` method. Recall that a strong orientation of a graph is a directed graph in which each undirected edge is given a direction such that the resulting directed graph is strongly connected. A strongly connected directed graph has a directed path from any vertex to any other vertex. Finally, Line 4 displays the strongly oriented directed graph `sgraph`, shown in Figure 4, with its vertices and directed edges.

An important question is: when can an undirected graph have an orientation such that the resulting directed graph is